JWT 게시판
구현 보고서 — 캐릭터 최적화 툴

웹서비스설계 01

서비스 소개와 구현 범위

학번 202212270 **이름** 최준혁 **제출일** 2026-10-28

서비스 이름 캐릭터 최적화 툴

기술 조합과 버전 React 19.3.0 (Vite 8.3.4) / Express 5.3.0, jsonwebtoken 9.0.3, sql.js 1.14.2 / Node.js v24.21.0, Windows 11 + WSL2 Ubuntu 24.04.4 LTS

GitHub 저장소 URL https://github.com/Zun-H-fighter/jwt-character-tool

소스 코드 기준 main 브랜치. 최종 제출 기준 커밋 SHA는 LMS에 제출합니다.

가상의 RPG를 플레이하는 사용자가 자신의 캐릭터(캐릭터 이름, 무기 이름·공격력, STR·DEX·INT·LUK)를 기록하고 전투력을 확인하는 도구입니다. 스탯이 높을수록 1포인트의 효과가 줄어드는 수확 체감 규칙을 두어, 다음 스탯 포인트를 어디에 투자하면 전투력이 가장 많이 오르는지 추천합니다. 누구나 캐릭터 목록과 상세를 볼 수 있고, 로그인한 사용자만 캐릭터를 등록할 수 있으며, 수정·삭제는 등록한 본인만 가능합니다.

기술 조합은 React와 Express를 선택했습니다. 두 쪽 모두 JavaScript라 화면에서 서버까지의 요청 흐름을 같은 언어로 따라가며 이해하기 쉽고, 제공 템플릿에서 인증 함수가 한 파일(`auth.js`)에 모여 있어 BE-01~03의 위치가 분명했기 때문입니다. 제공 템플릿에서 바꾼 범위는 인증·권한(`auth.js`), 프론트 통신(`api.js`), 화면(`main.jsx`)과 캐릭터 기능 파일 추가(`character.js`)이며, 게시글 API와 DB 구조는 그대로 유지했습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	backend/src/app.js	제공 코드 유지
로그인과 JWT 발급	완료	backend/src/auth.js `issueToken`	HS256, sub·iss·aud·exp
인증과 내 정보 조회	완료	backend/src/auth.js `currentUserId`	exp 존재·sub 형식·계정 존재 직접 확인
본인 글 작성·수정·삭제	완료	auth.js `assertOwner`, app.js 라우트	작성자는 토큰으로만 결정
만료·로그아웃·오류 표시	완료	frontend/src/api.js `request`, main.jsx `run`	401만 로그인 해제, 403은 유지
DB 데이터 유지	완료	backend/src/store.js (제공)	서버 재시작 후 유지 확인(T15)

기능	완료 여부	관련 파일	비고 또는 제한
캐릭터 기록·전투력 계산	완료	frontend/src/character.js	수치 범위 검사는 프론트에서만 수행

구조와 변경 내역

브라우저의 React 화면은 `/api`로 시작하는 요청을 Vite 서버(개발 5173, 빌드 확인 4173)로 보내고, Vite의 proxy가 이를 127.0.0.1:8000의 Express로 전달합니다. Express는 라우트에서 인증 함수(`auth.js`)와 입력 검사(`fields`)를 거친 뒤 `store.js`가 sql.js로 SQLite를 다루고, 변경할 때마다 `backend/data/board.sqlite` 파일로 저장합니다.

요청	흐름	토큰
로그인	main.jsx 로그인 폼 → api.js `login` → POST /api/auth/login → app.js 로그인 라우트 → users.js `verifyCredentials` → auth.js `issueToken` → 200 {accessToken, expiresIn, user}	응답의 accessToken을 api.js의 `session` 변수(브라우저 메모리)에 저장. localStorage·sessionStorage는 사용하지 않음
캐릭터 수정	main.jsx `save` → character.js `toPost` → api.js `api.update` → `request` → PUT /api/posts/{id} → app.js PUT route: `currentUserId`(인증) → `postId`·`getPost`(글 존재) → `assertOwner`(권한) → `fields`(입력) → `store.update`	api.js `authorizationHeaders`가 `Authorization: Bearer <JWT>`를 붙임. 공개 GET 목록·상세는 토큰 없이도 동작

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유 / 작성 출처
backend/src/auth.js `loadSecret`, `loadTtl`	없음	JWT_SECRET 64자리 hex 검사(빈 값·예시 키·형식 오류 시 시작 거부), 32바이트 변환, TTL 정수 검사	BE-01 서버 키 설정 / AI 초안을 직접 입력
backend/src/auth.js `issueToken`	501 NOT_IMPLEMENTED	HS256으로 sub·iss·aud·exp 발급	BE-01 / AI 초안을 직접 입력
backend/src/auth.js `currentUserId`	항상 1 반환(데모)	Bearer 추출, `jwt.verify`(HS256 고정, iss·aud), exp 존재·sub 형식·계정 존재 확인, 401 code 구분	BE-02 / AI 초안을 직접 입력, 입력 중 오류는 직접 수정
backend/src/auth.js `assertOwner`	검사 없음	작성자와 다르면 403 FORBIDDEN	BE-03 / AI 초안을 직접 입력
backend/src/app.js 에러 미들웨어, 시작 문구	500을 응답만 함	500일 때 `console.error`로 서버 로그 출력, 시작 문구 변경	디버깅용 / AI 제안 후 직접 추가
frontend/src/api.js `login`, `authorizationHeaders`, `request`, `canEdit`	TODO	메모리 세션 저장, 토큰 있을 때만 Bearer, 401만 세션 해제와 code별 안내, 204 미파싱, 작성자 비교	FE-01~04 / AI 초안을 직접 입력
frontend/src/main.jsx	게시판 화면	캐릭터 입력 폼·스탯 표·전투력·투자 추천, 404 처리, 비로그인 안내, 로그아웃 시 폼 초기화, 배너 교체	주제 기능, FE-03·04 / AI 제공 코드 반영 후 일부 직접 수정

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유 / 작성 출처
frontend/src/character.js (추가)	없음	캐릭터 ↔ title·content 변환, 입력 검사, 전투력·수확 체감·투자 추천 계산	API 명세 유지 / v1은 AI 제공 코드, v2 공식 변경은 직접 수정
tools/manual_tests.py (추가)	없음	T01~T17 API 시나리오 자동 실행과 기록	테스트 증거 / AI 제공 코드 그대로 사용

로컬 실행 기록

Windows 11의 WSL2 Ubuntu 24.04에서 Node.js v24.21.0으로 실행했습니다. 백엔드는 `backend/`에서 `npm ci` → `.env` 작성(새 JWT_SECRET) → `npm start`, 프론트는 `frontend/`에서 `npm ci` → `npm run build` → `npm run preview`로 실행하고 http://127.0.0.1:4173 에서 확인했습니다. SQLite 파일은 `backend/data/board.sqlite`이며 자동 생성됩니다. 자세한 절차는 저장소 README에 있습니다.

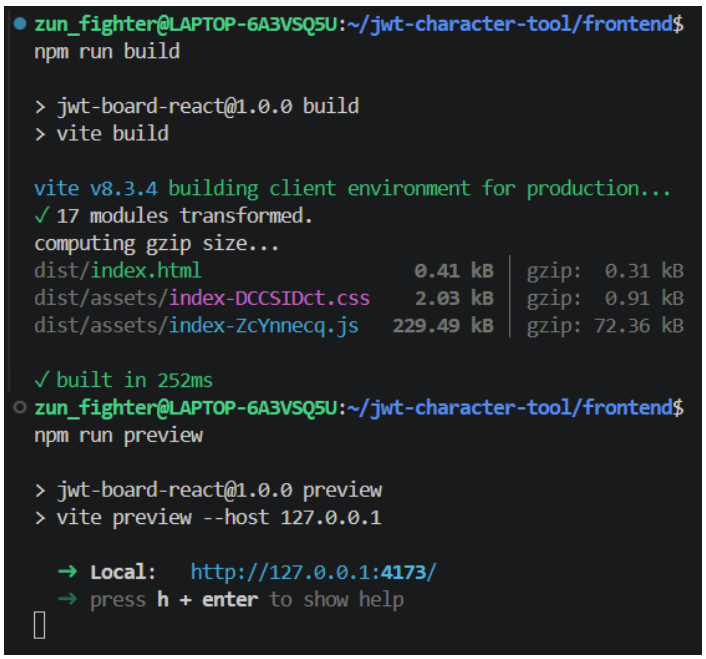


그림 1. 프론트 빌드(npm run build) 성공과 preview(4173) 실행 (docs/evidence/step7/build.png)

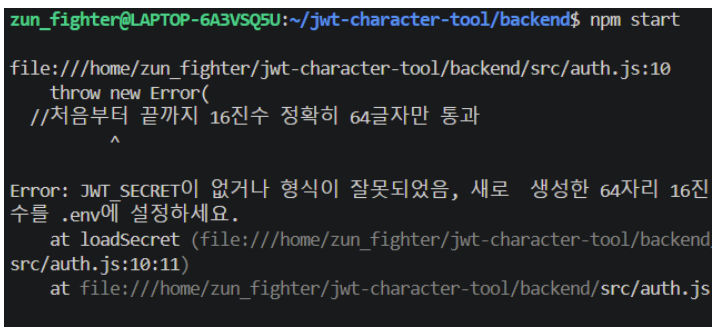


그림 2. JWT_SECRET이 없거나 형식이 틀리면 서버가 시작을 거부함 (docs/evidence/step1- 키 없거나 예시키거나 이상한 형식일때.png)

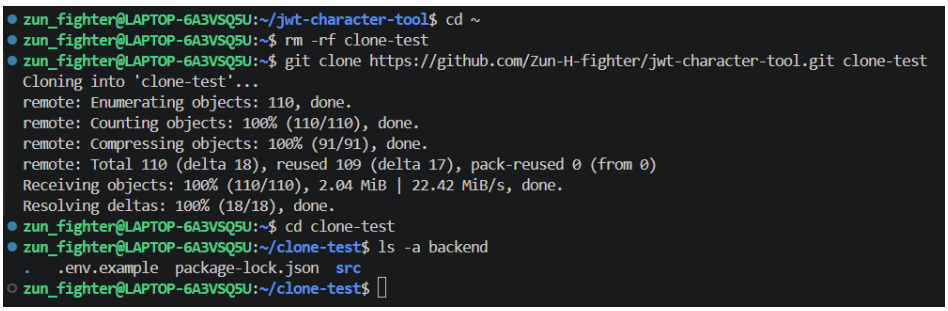


그림 3. 새 폴더(~/clone-test)에 clone — backend에 .env, node_modules, data가 없음 (docs/evidence/local_test/git clone and ls -a.png)

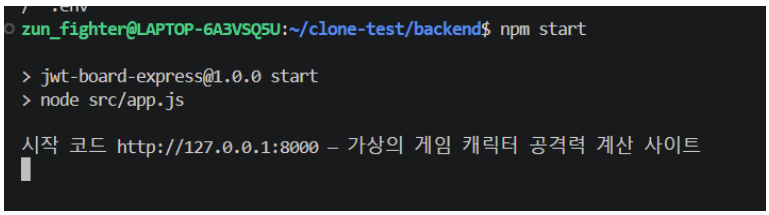


그림 4. clone한 폴더에서 새 JWT_SECRET 설정 후 백엔드 실행 (docs/evidence/local_test/npm start.png)

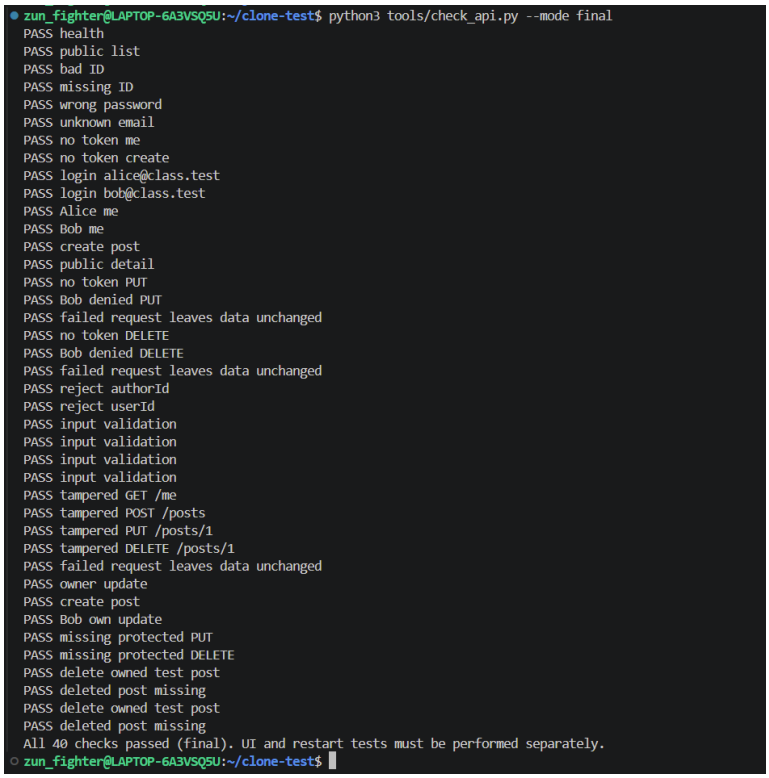


그림 5. clone한 폴더에서 check_api final 40개 통과 (docs/evidence/local_test/check_api_allpass.png)

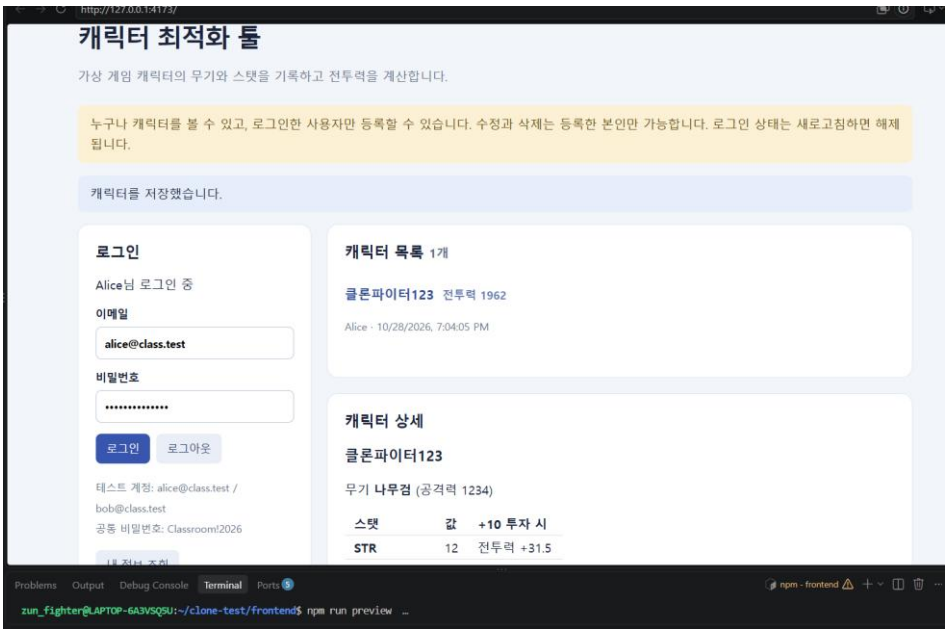


그림 6. clone한 폴더의 빌드본(4173)에서 로그인과 캐릭터 등록 (docs/evidence/local_test/local_4173_webbrowser.png)

핵심 코드 설명

3.1 로그인과 JWT 발급

파일 / 함수 backend/src/auth.js — loadSecret, issueToken (호출: backend/src/app.js 로그인 라우트 45~60행)

backend/src/auth.js

```
 9  function loadSecret() {
10    const hex = process.env.JWT_SECRET;
11    if (!hex || hex === SAMPLE_SECRET || !/^[0-9a-fA-F]{64}$/.test(hex)) {
12      throw new Error(
13        "JWT_SECRET이 없거나 형식이 잘못되었음, 새로  생성한 64자리 16진수를 .env에 설정하세요."
14      );
15    }
16    return Buffer.from(hex, "hex"); // 64자리 hex -> 32 바이트
17  }
```

backend/src/auth.js

```
27  const SECRET = loadSecret();
28  const TTL = loadTtl();
29  const ISSUER = process.env.JWT_ISSUER || "webservice-class";
30  const AUDIENCE = process.env.JWT_AUDIENCE || "board-api";
31
32  export function issueToken(user) {
33    // BE-01: 서버 환경변수 키로 HS256 JWT 발급. sub, exp, iss, aud 필수.
34    return jwt.sign({}, SECRET, { // payload는 비워도 됨, 필요한 클레임 전부 옵션으로 넣음
35      algorithm: "HS256",
36      subject: String(user.id), // sub: 문자열 사용자 ID
37      issuer: ISSUER,           // iss
38      audience: AUDIENCE,       // aud
39      expiresIn: TTL            // exp
40    });
41  }
```

입력 로그인 라우트가 fields로 email(1~254자)·password(1~128자) 형식을 검사하고, 제공 함수 verifyCredentials가 scrypt 해시로 비밀번호를 비교합니다. 형식 오류는 400 BAD_INPUT, 틀린 비밀번호와 없는 계정은 모두 401 INVALID_CREDENTIALS이며 이때 issueToken까지 가지 않으므로 토큰이 발급되지 않습니다.

처리 키는 서버가 시작할 때 loadSecret이 한 번 읽습니다. 처음부터 끝까지 16진수 64글자인 값만 통과시키고 32바이트로 변환하며, 빈 값·예시 키·형식 오류면 예외를 던져 서버가 시작하지 않습니다(그림 2). issueToken은 비밀번호나 해시 없이 사용자 ID만 sub(문자열)로 넣고, iss=webservice-class, aud=board-api, exp=발급 시각+TTL(기본 90초)을 HS256으로 서명합니다.

출력 200과 {accessToken, expiresIn, user:{id,name,email}}.

3.2 브라우저 토큰 저장과 전송

파일 / 함수 frontend/src/api.js — session, authorizationHeaders, request, login

frontend/src/api.js

```
1  //로그인 정보는 이 변수(browser memory)에만 보관!!
```

```
let session = { accessToken: "", user: null };
export const getSession = () => session;
export function clearSession(){
  session = { accessToken: "", user: null };
}

// FE-02: 토큰 있을때만 Authorizaiton 헤더 만들기!
function authorizationHeaders() { // 삼항 연산자로 토큰 있을때만 헤더 객체 생성, 없으면 빈 객체 {}!!
  return session.accessToken
    ? { Authorization: `Bearer ${session.accessToken}` }
    : {};
}
```

frontend/src/api.js

```
  const data =
    response.status === 204 ? null : await response.json().catch(() => null);
  if (!response.ok) {
    //FE-03: 401이면 메모리의 로그인 상태 해제!!, 403은 해제 안함.
    if (response.status === 401) clearSession(); // 401이면 강제 로그아웃
    const code = data?.code;
    const message =
      (response.status === 401 && AUTH_MESSAGES[code]) ||
      data?.message ||
      `요청이 실패했습니다. (HTTP ${response.status})`;
```

frontend/src/api.js

```
export async function login(email, password) {
  const data = await request("/auth/login", { //실패하면 request안에서 thorw!!
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user }; // 이 줄은 성공했을때만 실행
  return data.user;
}
```

로그인 응답 처리 `login`은 `request`로 로그인 요청을 보내고, 실패하면 `request` 안에서 예외가 던져져 57행이 실행되지 않으므로 실패를 성공으로 표시하지 않습니다. 성공하면 토큰과 사용자를 `session` 변수에 저장하고 화면에 "Alice님 로그인 중"을 표시합니다.

저장 위치 브라우저 메모리의 모듈 변수입니다. localStorage에 두면 같은 페이지의 모든 스크립트가 읽을 수 있어 스크립트 삽입 공격 시 토큰이 유출되기 쉽습니다. 대신 새로고침하면 로그인이 해제되며, 이는 과제 요구사항상 정상입니다.

Authorization 헤더 모든 요청이 `request` 하나를 지나가므로 `authorizationHeaders`가 /me·등록·수정·삭제에 자동으로 `Bearer <JWT>`를 붙입니다(그림 9). 토큰이 없으면 빈 객체를 돌려줘 `Bearer undefined` 같은 헤더를 만들지 않습니다.

로그아웃과 오류 로그아웃 버튼은 `clearSession`으로 토큰·사용자를 지우고 화면 상태를 즉시 갱신합니다. 응답이 401이면 code와 관계없이 `clearSession`을 호출하고 INVALID_CREDENTIALS/TOKEN_REQUIRED/INVALID_TOKEN/TOKEN_EXPIRED별 안내를 보여 줍니다. 403은 인증은 유효하고 권한만 없는 상태이므로 로그인을 유지합니다. 204는 본문이 없어 JSON으로 읽지 않습니다.

3.3 서버 JWT 검증

파일 / 함수 backend/src/auth.js — currentUserId, invalidToken

backend/src/auth.js

```
47  export function currentUserId(req) {
48    //BE-02: Bearer 추출 → 서명/알고리즘/만료/클레임 검증 → 실제 사용자 확인.
49    const header = req.get("Authorization") || "";
50    const match = /^Bearer\s+(\S+)$/i.exec(header); // Bearer 토큰 추출(중요)
51    if (!match) {
52      throw new ApiError(401, "TOKEN_REQUIRED", "로그인이 필요합니다");
53    }
54
55    let payload;
56    try {
57      payload = jwt.verify(match[1], SECRET, { // 태그 재계산
58        algorithms: ["HS256"],
59        issuer: ISSUER,
60        audience: AUDIENCE
61      });
62    } catch (err) {
63      if (err.name === "TokenExpiredError") { //서명은 진짜인데 시간만 지났다는 의미의 오류
64        throw new ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요");
65      }
66      throw invalidToken(); // 서명 불일치, 형식 오류, iss/aud 불일치 등
67    }
68
69    // jsonwebtoken은 exp가 없으면 그냥 통과시키므로 직접 확인!!(중요)
70    if (typeof payload.exp !== "number") throw invalidToken();
71    if (typeof payload.sub !== "string" || !/^[1-9]\d*$/.test(payload.sub)) throw invalidToken();
72
73    const userId = Number(payload.sub);
74    if (!findPublic(userId)) throw invalidToken(); // 실제 계정 여부 확인
75    return userId;
76  }
```

형식 확인 헤더가 없거나 `Bearer <값>` 형식이 아니면 401 TOKEN_REQUIRED입니다. `Bearer abc`처럼 형식은 맞지만 JWT가 아니면 다음 단계에서 INVALID_TOKEN이 됩니다.

서명·알고리즘·만료·클레임 `jwt.verify`는 `header.payload`로 HMAC-SHA256 태그를 서버 키로 다시 계산해 토큰의 태그와 비교하고, `exp` 만료와 `iss`·`aud` 값을 확인합니다. 허용 알고리즘을 HS256 하나로 고정해, 클라이언트가 조작할 수 있는 헤더의 `alg`로 검증 방식이 바뀌지 않게 했습니다. 이 라이브러리는 서명을 먼저 확인한 뒤 만료를 보므로 `TokenExpiredError`는 정상 서명의 만료 토큰에만 발생하며 TOKEN_EXPIRED로 응답합니다. 그 밖의 실패는 INVALID_TOKEN입니다.

라이브러리가 하지 않는 검사 jsonwebtoken은 `exp`가 없는 토큰을 만료 검사 없이 통과시키므로 70행에서 `exp` 존재를 직접 확인합니다. 발급 코드의 실수 등으로 만료 없는 토큰이 생겨도 영구히 유효한 토큰이 되지 않게 하기 위함입니다. 또 `sub`가 양의 정수 모양의 문자열인지, 그 ID의 계정이 실제로 있는지 확인합니다.

사용자 ID 전달 모든 검사를 통과한 숫자 ID만 반환하며, app.js는 이 값만 `/me`, 등록 작성자, 권한 비교에 사용합니다. 보호 API의 401에는 app.js 에러 미들웨어(103행)가 `WWW-Authenticate: Bearer`를 붙입니다.

3.4 작성자 권한 검사

파일 / 함수 backend/src/auth.js — assertOwner, backend/src/app.js — PUT·DELETE 라우트

backend/src/app.js

```
77  app.put("/api/posts/:id", (req, res) => {
78    const userId = currentUserId(req),
79      id = postId(req),
80      p = getPost(id);
```

```
  assertOwner(userId, p.authorId);
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
});
```

backend/src/auth.js

```
export function assertOwner(userId, authorId) {
  //BE-03: 다른 작성자이면 403 FORBIDDEN.
  if (userId !== authorId) {
    throw new ApiError(403, "FORBIDDEN", "본인 작성 글만 수정/삭제할 수 있습니다.");
  }
}
```

비교 대상 `userId`는 `currentUserId`가 토큰을 검증해 얻은 값이고, `authorId`는 `getPost`가 DB에서 읽은 글의 작성자입니다. 둘 다 클라이언트가 직접 바꿀 수 없는 값이며, 다르면 403 FORBIDDEN을 던져 `store.update·store.remove`에 도달하지 않으므로 DB가 바뀌지 않습니다.

401과 403 401은 요청한 사람이 누구인지 확인하지 못한 경우(토큰 없음·변조·만료)이고, 403은 누구인지는 확인했지만 그 글을 바꿀 권한이 없는 경우입니다. 라우트가 인증 → 글 존재 → 권한 → 입력 검증 → 변경 순서라서, Bob이 Alice 글에 공백 제목을 보내면 입력 오류(400)보다 권한 오류(403)가 먼저 나옵니다(T14).

작성자 위조 방지 등록 라우트는 작성자를 요청 본문이 아니라 `currentUserId`의 결과로 저장하고, `fields`가 title·content 외의 필드(authorId, userId 등)를 400 BAD_INPUT으로 거부합니다(T12).

테스트 결과와 증거

실행 환경 Windows 11 + WSL2 Ubuntu 24.04.4 LTS / Node.js v24.21.0 / Chrome 156 / 백엔드 `npm start`, 프론트 `npm run build` + `npm run preview`(4173)

로컬 배포 확인 새 폴더(~/clone-test)에 저장소를 clone한 뒤 README 절차대로 npm ci, .env 작성(새 JWT_SECRET), npm start, 프론트 npm ci → build → preview를 실행함. data/board.sqlite가 자동 생성되고, health 200, check_api final 40개 통과, 4173에서 로그인과 캐릭터 등록을 확인함(그림 3~6)

테스트 일시 및 코드 버전 2026-10-27~28, main 브랜치

토큰 TTL 일반 테스트 90초 / 만료 테스트 3초(`TOKEN_TTL=3 npm start`)

API 요청은 `tools/manual_tests.py`로 실행했습니다. 이 스크립트는 각 요청의 예상과 실제 HTTP·code를 비교하고, 실패한 변경 요청 뒤에는 글을 다시 조회해 원문이 그대로인지 확인하며, 결과를 `docs/evidence/api-tests.log`에 기록합니다(토큰은 앞 10글자만 남기고 가림). 화면 관련 항목은 브라우저에서 직접 확인했습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
T01	토큰 없이 목록, 존재하는 글 상세	200	200 / 200	PASS	api-tests.log
T02	Alice 로그인	200, 토큰·사용자, 화면 표시	200, user {id 1, Alice}, "Alice님 로그인 중"	PASS	log, 그림 7
T03	틀린 비밀번호, 없는 이메일	401 INVALID_CREDENTIALS	401 INVALID_CREDENTIALS ×2, accessToken 없음	PASS	log
T04	토큰 없이 /me, 등록, Basic 헤더	401 TOKEN_REQUIRED	401 TOKEN_REQUIRED ×3, WWW-Authenticate: Bearer	PASS	log
T05	Alice 작성 후 /me	201 authorId 1, /me Alice	201, Location 있음, authorId 1 / 200 Alice	PASS	log, 그림 9
T06	Alice 본인 수정·삭제	200, 204	200(작성자·createdAt 유지) / 204 본문 0바이트 / 재조회 404	PASS	log, 그림 10
T07	Bob 토큰으로 Alice 글 수정	403 FORBIDDEN, 원문 유지	403 FORBIDDEN, 재조회 원문 유지	PASS	log, 그림 11
T08	Bob 토큰으로 Alice 글 삭제	403 FORBIDDEN, 글 유지	403 FORBIDDEN, 재조회 원문 유지	PASS	log
T09	서명 첫 글자 변경 토큰으로 /me·등록·수정·삭제	401 INVALID_TOKEN, 변경 없음	401 INVALID_TOKEN ×4, 글 수·원문 유지	PASS	log
T10	TTL 3초 발급 5초 후 /me·수	401 TOKEN_EXPIRED,	401 TOKEN_EXPIRED ×4, 원문 유	PASS	log, 그림 12

ID	수행 절차와 사용자	예상 결과	실제 HTTP와 code	판정	증거
	정·삭제·등록	화면 해제	지, 화면 로그아웃·만료 안내		
T11	로그아웃 후 /me	헤더 없음, 401 TOKEN_REQUIRED	Authorization 헤더 없음, 401	PASS	log, 그림 13
T12	Bob 등록에 authorId:1 / userId:1 추가	400 BAD_INPUT, 미생성	400 BAD_INPUT ×2, 글 수 변화 없음	PASS	log
T13	본인 글에 빈 제목·공백 본문·101자·5001자	400 BAD_INPUT, 변경 없음	400 BAD_INPUT ×4, 원문 유지	PASS	log
T14	없는 ID 조회·수정·삭제, abc 조회, 겹친 조건	404 / 400	404 NOT_FOUND ×3, 400 BAD_INPUT, 무토큰 401, Bob 공백 제목 403	PASS	log
T15	글 저장 → 백엔드 종료·재실행 → 조회	글·작성자·내용 유지	200, 제목·내용·작성자·시간 동일	PASS	log, 그림 14
T16	로그인 후 새로고침	글 유지, 로그인 해제	목록 유지, "로그아웃 상태"	PASS	그림 15
T17	무토큰 수정·삭제, Bob 작성·수정·삭제	401 / 201·200·204	401 TOKEN_REQUIRED ×2 / 201 authorId 2, 200, 204	PASS	log

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP와 code	증거 위치
정상 조회·로그인·수정	200	200 (T01, T02, T05 /me, T06, T17)	api-tests.log
새 글 등록	201 및 Location	201, Location: /api/posts/{id} (T05, T17)	api-tests.log
본인 글 삭제	204 및 빈 본문	204, 본문 0바이트 (T06, T17)	api-tests.log, 그림 10
입력 오류·추가 작성자 필드	400 BAD_INPUT	400 BAD_INPUT (T12, T13, T14)	api-tests.log
잘못된 계정	401 INVALID_CREDENTIALS	401 INVALID_CREDENTIALS (T03)	api-tests.log
토큰 없음·헤더 형식 오류	401 TOKEN_REQUIRED	401 TOKEN_REQUIRED (T04, T11, T14, T17)	api-tests.log, 그림 13
변조·클레임 오류	401 INVALID_TOKEN	401 INVALID_TOKEN (T09)	api-tests.log
정상 서명의 만료 토큰	401 TOKEN_EXPIRED	401 TOKEN_EXPIRED (T10)	api-tests.log, 그림 12
다른 작성자의 변경	403 FORBIDDEN	403 FORBIDDEN (T07, T08, T14)	api-tests.log, 그림 11
없는 글	404 NOT_FOUND	404 NOT_FOUND (T06 삭제 후, T14)	api-tests.log

WWW-Authenticate 보호 API의 401 응답(T04, T09, T10, T11, T17)에서 `WWW-Authenticate: Bearer`를 스크립트로 확인했고, 브라우저 만료 화면(그림 12)의 Response headers에서도 확인했습니다.

401과 403의 화면 차이 401을 받으면 `request`가 세션을 지워 로그인 카드가 "로그아웃 상태"로 바뀌고 재로그인을 안내합니다(그림 12). 403은 세션을 지우지 않아 로그인 상태가 유지되며, 화면에서는 타인 글의 수정·삭제 버튼 자체가 표시되지 않습니다(그림 11).

데이터 불변 T07~T09, T10, T13, T14의 실패한 변경 요청 뒤 해당 글을 공개 조회해 처음 조회한 내용과 모든 필드가 같음을 확인했습니다. 실패한 등록 요청 뒤에는 목록의 글 수가 변하지 않았습니다.

겹친 조건 토큰 없이 없는 글(999999) 수정은 401 TOKEN_REQUIRED, 정상 토큰으로 같은 글 수정은 404 NOT_FOUND였습니다(T14).

자동 검사 `python3 tools/check_api.py --mode final`은 40개, `TOKEN_TTL=3` 서버에서 `--expiry`를 붙인 검사는 48개를 모두 통과했습니다(그림 8).

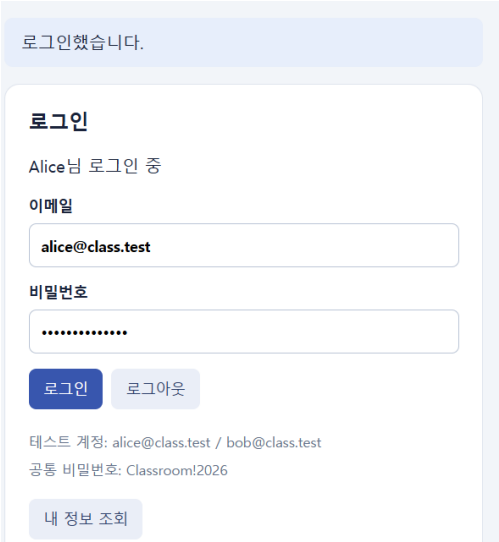


그림 7. Alice 로그인 후 화면의 로그인 사용자 표시 (T02) (docs/evidence/frontend_part/step-5 alice님 로그인중.png)

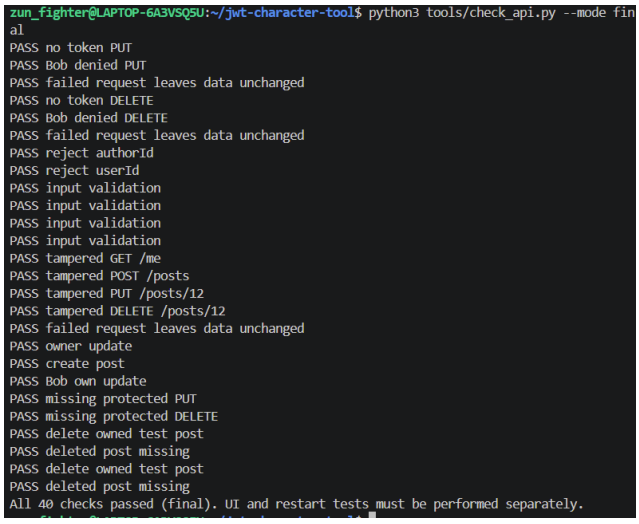


그림 8. 제공 자동 검사 final 모드 40개 통과 (docs/evidence/step7/check_api final 통과!!.png)

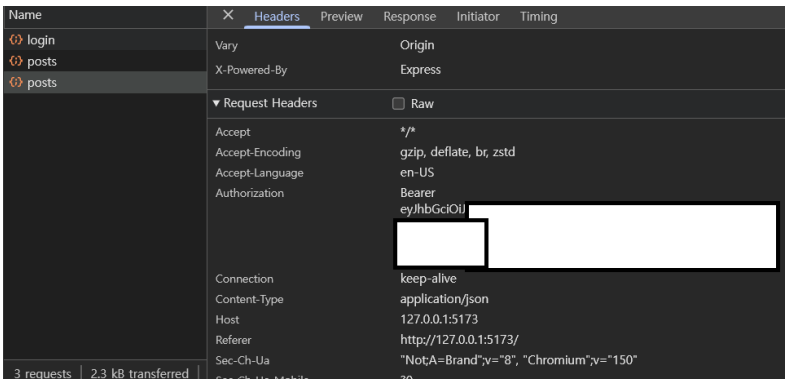


그림 9. 증거 1: 글 등록 요청에 Authorization: Bearer 헤더가 첨부됨 (토큰 원문은 가림) (docs/evidence/frontend_part/step5 글 등록시 리퀘스트 헤더에 authorizaton: bearer 확인.png)

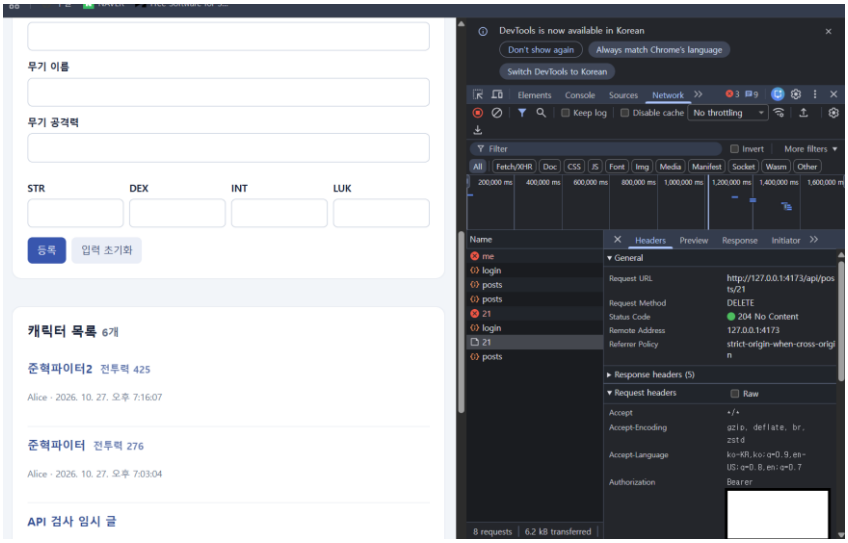


그림 10. 본인 글 삭제 204 No Content (토큰은 가림) (docs/evidence/step7(browser)/T06-delete-204.png)



그림 11. 증거 2: Bob 로그인 상태에서 Alice 캐릭터 상세 — 수정·삭제 버튼 없음. 서버 403은 T07·T08 기록 참고 (docs/evidence/step7(browser)/T07-bob-no-buttons.png)

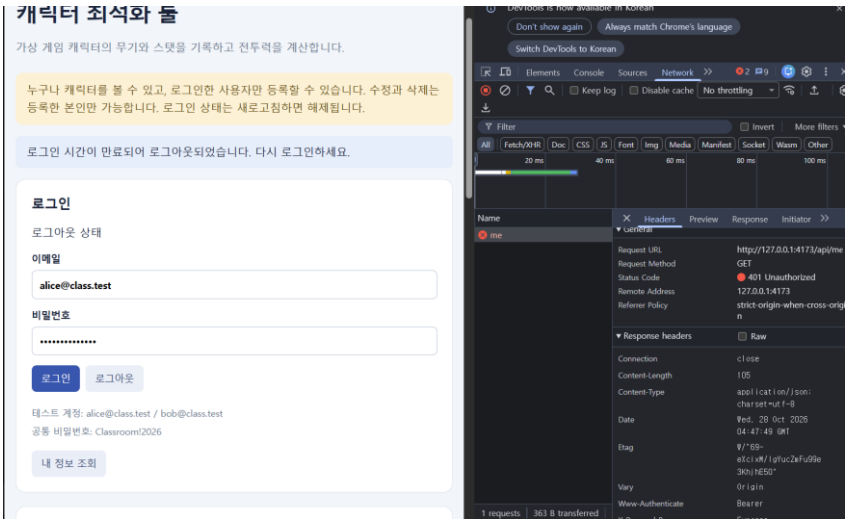


그림 12. 증거 3: 만료 토큰으로 /me 요청 — 401, WWW-Authenticate: Bearer, 화면 로그아웃과 만료 안내 (T10) (docs/evidence/step7/만료토큰 내정보조회시도.png)

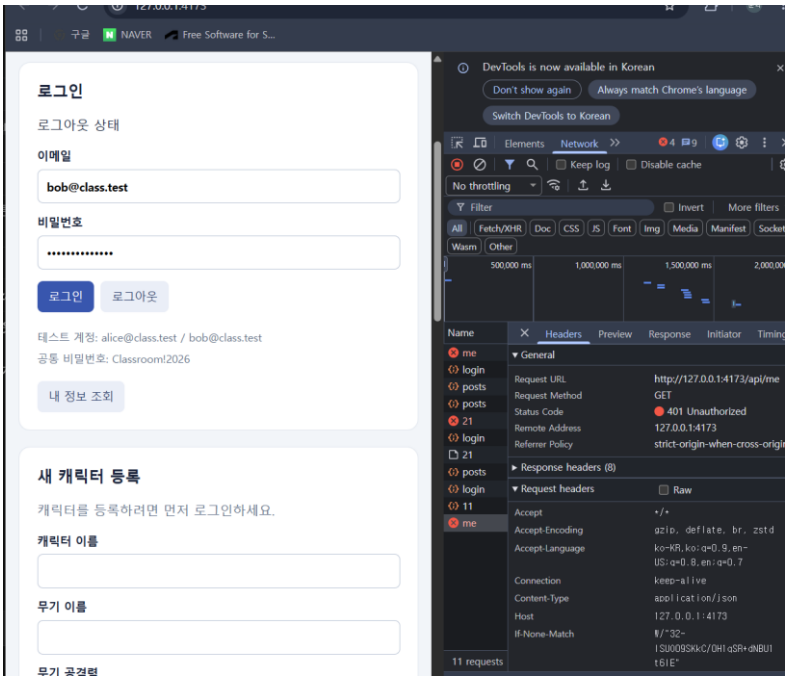


그림 13. 로그아웃 후 /me — Authorization 헤더 없이 401 (T11) (docs/evidence/step7(browser)/T11-logout.png)

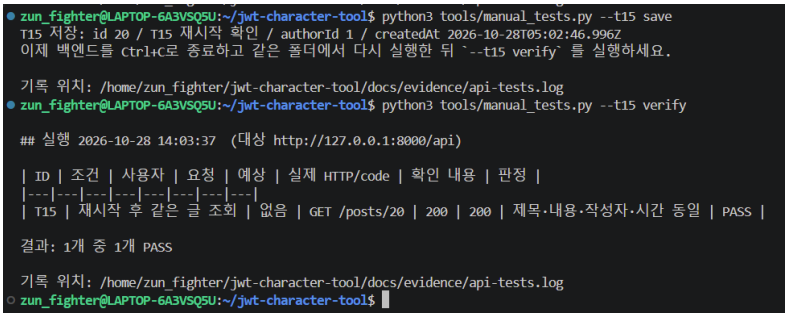


그림 14. 백엔드 재시작 후 같은 글 조회 PASS (T15) (docs/evidence/step7/restart 후 같은 글 조회 pass.png)

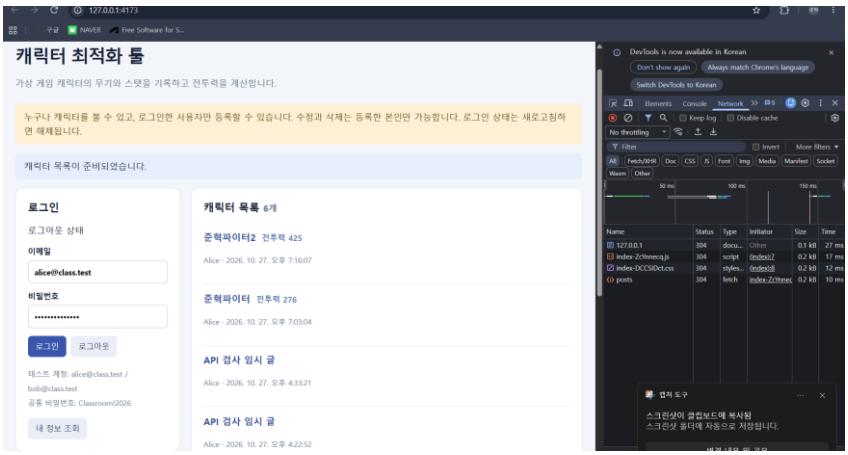


그림 15. 새로고침 후 목록은 유지되고 로그인은 해제됨 (T16) (docs/evidence/step7(browser)/T16-refresh.png)

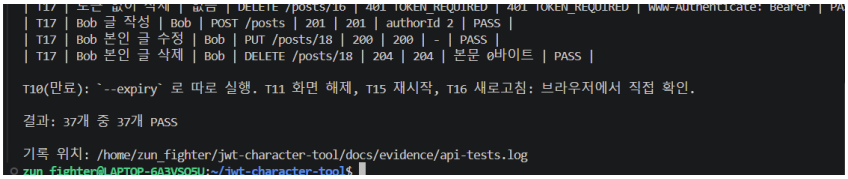


그림 16. T01~T17 시나리오 스크립트 실행 결과 37개 PASS (docs/evidence/step7/manual_check all pass.png)

AI 활용과 참고 자료

AI 사용 여부 사용. 단계별로 그때그때 `docs/ai-log.md`에 기록했으며, 아래는 그 요약입니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude (Anthropic) / claude-opus-5-5로 설정된 세션	학습: 과제 문서 분석, 요청 흐름(화면→api.js→라우트→auth.js→store.js) 설명	전체 구조 이해	참고
같음	설계: 캐릭터 정보를 content에 JSON으로 저장해 API 명세 유지	frontend/src/character.js	수정 후 반영
같음	생성: 키 검사, 발급·검증·권한 코드	backend/src/auth.js	제안 코드를 직접 입력, 입력 중 오류는 직접 수정
같음	생성: 로그인 연결, 헤더 첨부, 응답 코드 처리	frontend/src/api.js, main.jsx	제안 코드를 직접 입력, 주석은 본인 작성
같음	생성: 캐릭터 화면 v1, 테스트 스크립트	main.jsx, character.js, tools/manual_tests.py	그대로 반영(동작은 테스트로 검증)
같음	디버깅: 500 오류, /post 오타, mainstat 오타 원인 찾는 방법	auth.js, api.js, character.js	원인 확인과 수정은 본인
같음	코드 검토: 환경변수 이름 오타로 aud가 명세와 다르게 발급됨을 지적	auth.js 29~30행	수정 후 반영
같음	문장정리: README, 보고서 초안	README.md, 본 보고서	초안 검토 후 수정 반영

대표 사례 1 — 변조 토큰에 401 대신 500 이 나온 오류

도구 / 날짜 Claude / 2026-10-27

요청한 내용 BE-01~03 구현 코드와 설명. 이후 check_api에서 `FAIL tampered GET /me: expected HTTP 401, got 500`이 나와 원인을 질문

AI가 제안한 내용 app.js 에러 미들웨어가 ApiError가 아닌 예외를 모두 500으로 바꾸므로, catch 블록에서 ApiError가 아닌 것이 던져진 것이라고 범위를 좁힘. 원인을 직접 보기 위해 500일 때 `console.error(error)`를 추가하도록 제안

실제 반영 위치 `backend/src/app.js` 102행, `backend/src/auth.js` `invalidToken`(43~45행)

내가 확인하고 수정한 내용 서버 로그에서 `ReferenceError: invalidToken is not defined`를 확인함(그림 17). 함수를 선언했는데도 이 오류가 난 이유는 `invalidToken`을 `issueToken` 함수 안에 넣어서 다른 함수에서 보이지 않았기 때문이었고, 바깥으로 옮겼음. 재검사에서는 `code` 자리에 안내 문구가 들어간 것을 발견

했는데, `new ApiError(401, 문구)`로 인자 하나를 빠뜨려 문구가 두 번째 인자인 code로 들어간 것이어서 `"INVALID_TOKEN"`을 추가함. 정상 경로만 시험했다면 이 오류를 모르고 지나갔을 것

검증 방법과 결과 `check_api.py --mode final` 재실행 → 40개 통과(그림 8)

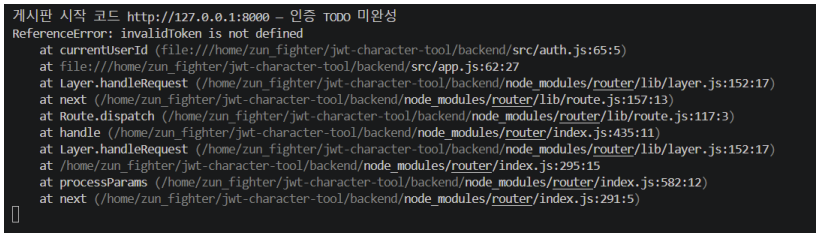


그림 17. 500 원인을 서버 로그로 확인 — ReferenceError: invalidToken is not defined (docs/evidence/backend_part/step 2-4 오류 수정을 위한 로그분석.png)

대표 사례 2 — 전투력 공식 개선(수확 체감)

도구 / 날짜 Claude / 2026-10-27

요청한 내용 처음 공식 `무기 공격력 × (1 + 주스탯/100)`은 본인이 정했으나, 주스탯만 반영하면 최적화 의미가 없어서 부스탯도 반영하고 스탯이 높을수록 효율이 떨어져 낮은 스탯 투자가 유리하도록 바꾸자고 요청(본인 아이디어)

AI가 제안한 내용 유효 스탯 `200×스탯/(스탯+200)`, 전투력 `무기 공격력 × (1 + (주스탯 유효값 + 0.3×부스탯 유효값 합)/100)`, 스탯별 +10 투자 효과 표. 반올림된 전투력끼리 빼면 증가량 차이가 사라져 반올림 전 값으로 비교하도록 수정

실제 반영 위치 `frontend/src/character.js` `effective`, `rawPower`, `combatPower`, `gainFor`, `bestInvestment`, `main.jsx` 스탯 표

내가 확인하고 수정한 내용 파일을 받지 않고 수정 위치 7곳을 안내받아 직접 수정함. 입력 중 `mainStat`을 `mainstat`으로 잘못 써서 등록 후 화면 전체가 하얗게 되는 오류가 났고, 브라우저 콘솔의 스택 트레이스(character.js:87)로 위치를 찾아 수정함

검증 방법과 결과 STR 300·DEX 20·INT 5·LUK 10, 공격력 120 입력 시 전투력 276, 가장 낮은 INT 추천(+3.3)을 확인. INT 500인 캐릭터에서 DEX 추천(+4)을 확인(그림 11)

대표 사례 3 — 자동 검사가 놓친 aud 오류

도구 / 날짜 Claude / 2026-10-28

요청한 내용 보고서 작성을 위해 커밋된 저장소 전체를 검토 요청

AI가 제안한 내용 `auth.js`가 `process.env.JWT_ISSuer`, `process.env.AUDIENCE`를 읽어 .env의 `JWT_ISSUER`, `JWT_AUDIENCE`가 반영되지 않고, AUDIENCE 기본값이 `borad-api`로 오타여서 발급 토큰의 aud가 명세(board-api)와 다르다고 지적. 발급과 검증이 같은 잘못된 값을 써서 서로는 맞고, check_api는 aud 값을 보

지 않아 통과했던 것

실제 반영 위치 backend/src/auth.js 29~30행

내가 확인하고 수정한 내용 환경변수 이름을 JWT_ISSUER, JWT_AUDIENCE로, 기본값을 board-api로 수정함.

검증 방법과 결과 수정 후 서버를 재시작하고 manual_tests.py를 다시 실행해 37개 모두 PASS 확인.

Aud: 'board-api' 확인도 디코드해서 했음.

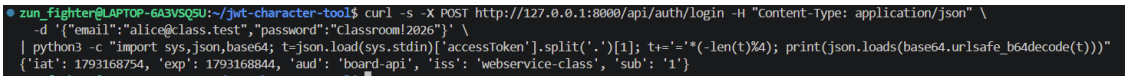


그림 18. 수정 후 발급 토큰의 payload를 디코드해 aud가 board-api임을 확인 (docs/evidence/step8/aud 확인.png)

기타 사용 내역

- 0 단계: 과제 문서 분석, 진행 순서 설계, 시작 코드의 요청 흐름 설명(학습)
- 1 단계: JWT_SECRET 검사 코드와 정규식, ES 모듈 import 순서와 dotenv 로드 시점 설명
- 5 단계: "API 경로가 없습니다" 문구가 app.js 의 미등록 경로 404 에서 나온다는 설명을 듣고, Network 의 Request URL 로 api.js 의 /post 오타를 직접 찾아 수정
- 7 단계: T01~T17 시나리오 스크립트(tools/manual_tests.py) 작성. 실행과 결과 확인은 본인 PC 에서 수행
- 8 단계: README 와 본 보고서 초안 작성. 이해도 확인 답변은 대화 중 본인이 답한 내용을 바탕으로 정리

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
수업 제공 시작 코드(backend-express, frontend-react)와 과제 문서	전체	제공 코드 유지 부분과 변경 부분을 2장 표로 구분
jsonwebtoken — https://github.com/auth0/node-jsonwebtoken	auth.js issueToken, currentUserId	jwt.sign, jwt.verify 옵션과 오류 이름(TokenExpiredError)
Vite — https://vite.dev/guide/	frontend/vite.config.js (제공)	개발·preview 프록시
Express — https://expressjs.com/	backend/src/app.js (제공)	에러 미들웨어 동작

이해도 확인과 회고

1. 요청 본문의 authorId를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

Body의 authorId를 믿으면 bob이 body를 조작해서 alice이름으로 글을 만들 수 있음.

내 코드에선 authorId를 currentUserId(req)가 토큰을 검증한 후 얻는 payload의 sub에서 ID를 추출해서 이런 authorId 위조시도를 막음. 또 fields()가 title, content 외의 필드(authorId, userId)가 오면 400 BAD_INPUT으로 거부함. T12에서 Bob이 authorId:1, userId:1을 넣은 요청이 각각 400이고 글이 만들어지지 않음을 확인했음.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

`canEdit`로 버튼을 숨기는 건 보안이 아니라 편의 기능임. 버튼만 숨긴 거라 개발자 도구나 curl로 요청을 직접 보내면 우회할 수 있음. 실제로 막는 건 백엔드 `auth.js`의 `assertOwner`이고, 토큰으로 확인한 사용자 ID와 DB에 저장된 글의 authorId를 비교해 다르면 403을 줌. T07·T08에서 Bob 토큰으로 직접 요청해 403과 원문 유지를 확인했음.

3. JWT decode와 verify의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

JWT의 payload는 암호화가 아니라 Base64url 인코딩이라 decode는 그냥 펴서 읽는 것이고 누구나 할 수 있음. verify는 HS256, 즉 HMAC-SHA256 MAC으로 `header.payload`의 태그를 서버 키로 다시 계산해 붙어 온 태그와 비교, 공격자가 sub를 "2"로 바꾸면 메시지가 바뀌어 태그가 맞지 않고, 키가 없으니 바뀐 메시지에 맞는 태그를 새로 만들 수 없음. 그래서 decode 결과만 믿으면 위조된 sub를 그대로 들어감. 내 코드는 `jwt.verify`에 허용 알고리즘을 HS256으로 고정해 공격자가 검증 알고리즘을 마음대로 정하지 못하게 하고, iss·aud·만료를 확인하며, 라이브러리가 exp 없는 토큰을 통과시키므로 exp 존재를 직접 확인한다. exp 확인이 없으면 실수로 만료 시간을 넣지 않은 토큰을 무한히 사용가능. 마지막으로 sub 형식과 실제 계정 존재를 확인함.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃하면 탭의 메모리에 있는 api.js의 session 변수들을 다 지워버림, 여기에 토큰과 사용자 정보도 껴있어서 다음요청부터는 Authorization헤더가 빠져버림.(T11) 토큰 만료는 서버가 토큰 서명은 통과인데 exp가 만료되서 거부하는 것임. 프론트에서 401 TOKEN_EXPIRED를 받으면 clearSession()을 호출하고 재로그인 안내 메시지 띄움.(T10) 새로고침시 api.js의 let session = {..} 실행, {}안에 값이 빈 값이기 때문에 새로고침하면 세션은 빈 값으로 시작함, 게시글은 DB에 있는 글들이기 때문에 그대로 있음.(T16) 서버는 발급한 토큰 관리를 exp로만 하기 때문에 TTL시간동안 복사한 토큰을 쓴다면 사용 가능할 것임.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

변조 토큰에 401이 아니라 500이 나온 오류. 에러 미들웨어가 ApiError가 아니면 500으로 처리한다는 걸 알고 catch 블록 쪽을 의심했지만, 응답에는 원인이 안 보여서 500일 때 `console.error`를 추가해 서버 로그를 봤음. `ReferenceError: invalidToken is not defined`였는데 함수는 이미 있었고, 알고 보니 `issueToken` 안에 갇혀 있어서 다른 함수에서 안 보임. JS는 그 줄이 실행될 때 이름을 찾기 때문에 정상 토큰 테스트에서는 문제가 없었고 변조 토큰에서만 터졌음. 함수를 밖으로 옮긴 뒤 다시 돌리니 이번엔 code에 문구가 들어가 있었고, ApiError 인자를 하나 빠뜨려 위치가 밀린 것. 둘 다 고친 뒤 check_api를 다시 돌려 40개 통과를 확인했음.

현재 남은 문제와 다음에 개선할 점 캐릭터 수치(공격력·스탯) 범위 검사가 프론트에만 있어 API를 직접 호출하면 범위를 벗어난 값이 백엔드에 저장 될 수 있음. 공통 API 명세를 유지하려고 서버는 content를 문자열로만 검사해서 이런 구조임, 화면은 캐릭터 형식이 아닌 content를 일반 텍스트로 보여 주어 깨지지 않음. 명세를 바꿀 수 있다면 캐릭터 필드를 서버에서 검증하는 것이 맞는 구조임. 또 자동 검사를 통과해도 aud 값처럼 검사하지 않는 항목은 틀릴 수 있다는 점을 배웠으므로, 다음에는 발급한 토큰의 클레임을 직접 디코드해 명세와 비교하는 확인을 추가하는 방향으로 가보겠음.

제출 확인

[v] 실제 코드와 설명 일치 [v] 테스트 직접 수행 [v] AI 활용 및 출처 기록 [v] 비밀 키와 토큰 제거

GitHub 확인

[v] 루트 README 작성 [v] 보고서 PDF와 증거 업로드 [v] README 문서 링크 확인 [v] 교수자 접근 가능 [v] 저장소 URL과 커밋 SHA 제출